

AI Trip Planner

Complete Project Guide

Built with LLMs, LangChain, LangGraph, FastAPI & Streamlit

This guide covers, in simple words: the full Week 1-4 build journey, every real bug found and fixed, how the project works step by step, which tools/models are used and how they connect, plus tips, resume notes, and interview talking points.

Project Type	Agentic AI Travel Assistant (tool-using LLM agent)
Core Framework	LangChain + LangGraph
Interfaces	FastAPI (backend) + Streamlit (frontend)
LLM Provider	Groq (free) - deepseek-r1-distill-llama-70b
Cost	100% free to build and run (no paid APIs required)

Contents

1. Introduction - Why This Project Exists
2. Project Workflow (Step by Step Diagram)
3. Week 1 - Foundations & Setup
4. Week 2 - Building the Tools
5. Week 3 - LangGraph Agent Workflow
6. Week 4 - API, UI & Integration
7. All Bugs Found & Fixed (Simple Explanation)
8. Tools & Models Used (Step by Step Flow Diagram)
9. Tips
10. Resume Notes
11. Interview Talking Points

1. Introduction - Why This Project Exists

A normal AI chatbot only knows what it learned during training - it has no live internet access. It does not know today's weather, today's currency rates, or which restaurants are open right now. If you ask it directly, it either admits it does not know, or worse, confidently makes up a wrong answer (this is called a hallucination). For a trip planner, a wrong weather guess or a made-up hotel price is genuinely harmful, not just annoying.

The fix used in this project: give the AI model **tools** - small functions it can call mid-conversation to fetch real, live data - and let it decide, turn by turn, whether it needs to use one before answering. This is what turns a plain chatbot into an **agent**. This project, AI Trip Planner, is built entirely around that idea: an agent that plans full trips using live weather, live place search, live currency conversion, and guaranteed-correct budget maths - instead of guessing from memory.

What the finished project does

- Takes a plain request like *'Plan a 5-day trip to Goa'*
- Decides on its own which tools it needs to call to answer correctly
- Fetches live weather, live places/attractions, live currency rates, and calculates a real budget
- Returns a complete, structured itinerary in clean Markdown
- Runs 100% free - no paid API or platform required

2. Project Workflow (Step by Step Diagram)

This diagram traces one full request through the entire system - from the user typing a question in Streamlit, to the FastAPI backend, to the LangGraph agent deciding whether to call tools, to the final answer coming back and being displayed.

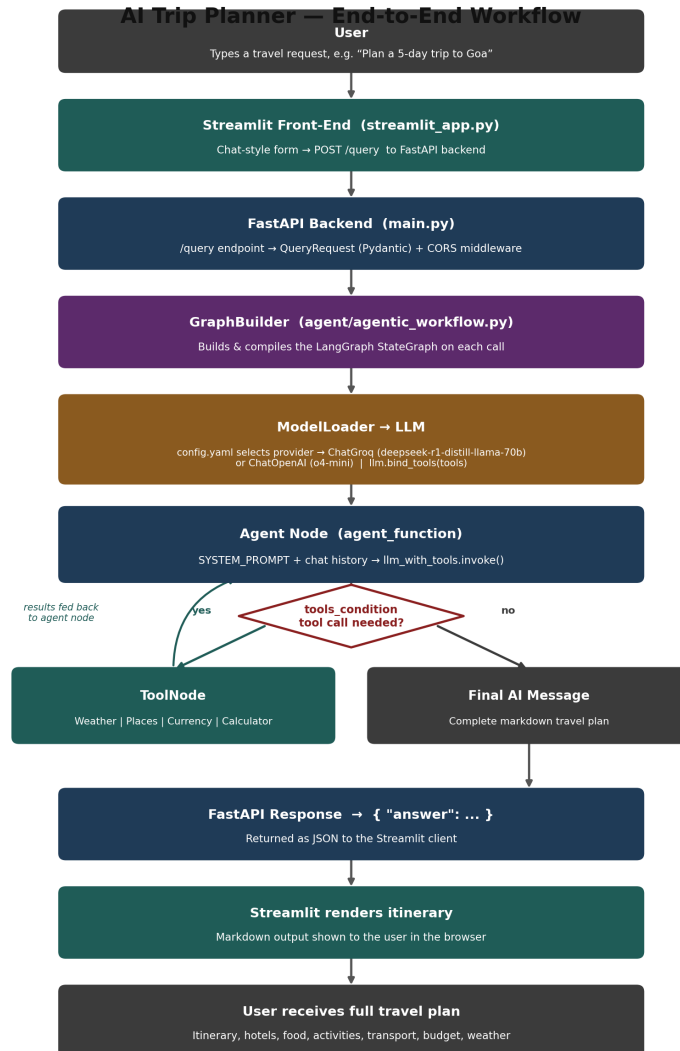


Figure 1: Full request/response workflow of the AI Trip Planner application.

WEEK 1 — Foundations & Setup

Week 1 is about getting the project skeleton right before writing any AI logic: a clean Python environment, an organized folder structure, centralized settings, and safe handling of secrets.

Environment setup: Python 3.10 + uv

The project was pinned to Python 3.10 and set up using **uv** instead of the classic pip + venv combo. A virtual environment is a private, isolated toolbox just for this project, so it never conflicts with other projects on the same computer. Pinning a specific Python version means the project keeps working the same way forever, instead of quietly breaking after a future Python update.

Why uv instead of pip/venv? Speed, mainly - uv installs the same packages much faster, and it combines Python-version pinning, environment creation, and package installing into one tool instead of three separate ones.

Important clarification: this environment setup and dependency install is a one-time step, done only in Week 1. Weeks 2-4 simply add new files into that same already-set-up environment.

Project folder structure

The codebase is split into folders, each with one clear job: `agent/` (the LangGraph brain), `tools/` (individual abilities), `utils/` (the plumbing that actually calls outside APIs), `prompt_library/` (AI instructions), `config/` (settings), plus `main.py` (FastAPI) and `streamlit_app.py` (UI). This is called **separation of concerns** - each file does one job, so bugs are easier to find, and pieces are easier to swap out later without breaking anything else.

Configuration file (config.yaml)

Instead of typing the model name directly into multiple Python files, it lives in exactly one place: `config.yaml`. Change one line there, and everything that reads from it follows automatically - no hunting through several files to keep them all in sync.

Secrets and API keys (.env file)

Every API key (Groq, Tavily, OpenWeatherMap, Exchange Rate) lives in a `.env` file, read using `python-dotenv` - never typed directly into code. This protects the keys if the code is ever shared publicly, and lets different people use their own private keys without touching the shared code.

Deciding: free project, not a paid one

A deliberate decision was made early: this project should run on completely free API tiers, no credit card required anywhere. Groq (LLM), Tavily (search), OpenWeatherMap (weather), and Exchange Rate API (currency) all offer genuine free access. Google Places was intentionally treated as optional, since it requires a Google Cloud billing account with a card on file - the project was built with a fallback (Tavily) so it never has to be used.

Exception handling & logging scaffolding

Empty placeholder folders (`exception/`, `logger/`) were created early, before any real feature existed, so that error handling and logging get added consistently in one place as features are built - instead of retrofitting it later into already-finished code.

WEEK 2 — Building the Tools

Week 2 gives the AI its 'hands' - a set of independent, tested tools that each fetch one kind of real world information.

The @tool decorator

Every tool function is written as a normal Python function with `@tool` placed above it. An AI model cannot literally run Python code - it only generates text. The `@tool` decorator builds a bridge: it describes the function to the model (its name, its purpose from the docstring, its expected input), so the model can ask for it to be called, and the real Python function actually runs and returns the result.

Two layers per tool: utils/ class + tools/ wrapper

Each feature is split into two files - a plain class in `utils/` that does the actual API work, and a thin wrapper in `tools/` that exposes it as an `@tool` function. This makes the core logic reusable in other projects, and makes each piece easy to test on its own.

The four tools built

- **Weather Info Tool** - current weather + forecast, from OpenWeatherMap. Live data is required because weather changes daily and the AI's memorized knowledge would be stale.
- **Place Search Tool** - attractions, restaurants, activities, transportation. Uses Google Places first, and automatically falls back to Tavily if Google is not configured or fails - this is what keeps the whole project free to run.
- **Currency Converter Tool** - live conversion rates from Exchange Rate API, since exchange rates change daily and cannot be trusted from the model's memory.
- **Expense Calculator Tool** - total cost, hotel cost, daily budget maths done in plain Python, not by the AI itself.

Why give the AI a calculator - can't it just do the maths?

A calculator function is 100% deterministic - $2000 * 5$ always equals 10000, every single time. An LLM doing maths 'in its head' is not actually calculating - it is predicting the most likely next number based on patterns it saw during training. It is usually right for simple numbers, but can make small, hard-to-notice mistakes with bigger or multi-step sums. Since this project is about budgets, the AI is left to decide *what* needs adding up, while the tool guarantees the adding up itself is always correct.

Testing tools before connecting to the AI

Every tool was tried by hand first inside a Jupyter Notebook, calling functions directly and checking the result - before ever wiring them into the LangGraph agent. This way, if something breaks later, it is much easier to know whether the problem is in a specific tool or in the AI's decision-making, instead of debugging everything blindly at once.

WEEK 3 — LangGraph Agent Workflow

Week 3 connects the tools from Week 2 to the LLM, and turns it into an actual decision-making agent using LangGraph.

ModelLoader

A small class reads `config.yaml` and returns the correct LLM object (ChatGroq or ChatOpenAI). This means every part of the app asks one single loader for 'the LLM', instead of each file separately writing its own model-building code - so switching providers only ever needs one change, in one place.

The system prompt

One long instruction block tells the model to always give two itinerary options (mainstream + off-beat), and always include hotels, attractions, restaurants, transport, a cost breakdown, a daily budget, and weather - in clean Markdown. Without this, an LLM gives a different structure every time it is asked. For a tool meant to be reliable, a consistent, predictable output format matters a lot.

`bind_tools()` - how the model learns the tools exist

`llm.bind_tools(tools)` is the line that actually teaches the model, at the API level, which functions it is allowed to request, what each one does (from the docstrings written in Week 2), and what input each one needs. Without this line, the LLM has zero awareness that any tools even exist.

Why LangGraph instead of a simple if/else script?

A simple script like 'if the question contains weather, call the weather tool' fails immediately for a real request like 'Plan a 5-day trip to Goa' - it does not mention weather, currency, or cost at all, yet the agent might still need all three tools, possibly more than once, in an order that cannot be predicted in advance. LangGraph instead builds a **loop**: ask the model what it wants to do next, run whatever tool it requests, feed the result back in, and ask again - repeating until it says it has a final answer. That decision-making loop is what makes it an agent, not a scripted chatbot.

StateGraph and MessagesState

'State' is simply everything that has happened so far in the conversation - every message, every tool call, every result - kept as a running list. Without tracking this, the agent would forget what tool it just called and what came back, by the time it is asked 'now what?' again.

The graph: agent node, tools node, conditional edge

An **agent node** runs the LLM. A **tools node** actually executes whichever tool was requested. A conditional check called `tools_condition` decides, after every agent step, whether to route to the tools node (if a tool was requested) or straight to END (if a final answer was given). After a tool runs, control loops back to the agent node with the fresh result.

Visualizing the graph

LangGraph can automatically draw the real, actual graph as a picture (a PNG file), using `draw_mermaid_png()`. This is a literal image with boxes and arrows showing exactly how the agent, tools, start, and end are connected in the real running code - not just an idea to imagine. Wiring mistakes (like a missing connection) are hard to spot in code, but instantly obvious as an odd-looking arrow in the picture.

Testing the full agent before building the API

Just like individual tools in Week 2, the entire compiled agent was tested directly in the notebook first - asking it a sample question and checking it correctly calls tools and gives a sensible final answer - before wiring it into FastAPI in Week 4.

WEEK 4 — API, UI & Integration

Week 4 turns the working agent into an actual app a real person can use, instead of something only runnable inside a notebook.

FastAPI backend

`main.py` exposes one endpoint, `POST /query`, that takes a question, builds the agent, runs it, and returns the answer as JSON. The backend (AI logic) and frontend (visual UI) are kept as two separate, independent pieces - this is the standard pattern almost every real app uses, and it means the same backend could later serve a mobile app or a different website without duplicating any AI logic.

QueryRequest (Pydantic) and CORS

A small `QueryRequest` class automatically checks that incoming requests contain valid text, rejecting bad data before it even reaches the AI logic. CORS middleware explicitly allows the Streamlit app to talk to the FastAPI backend - without it, the browser's own security rules would silently block the connection, even though both sides of the code are correct.

Streamlit frontend

`streamlit_app.py` is a simple chat-style screen written entirely in Python - no HTML, CSS, or JavaScript needed. It sends the user's question to the FastAPI backend using a normal HTTP request (the `requests` library), and displays the returned itinerary.

Exporting the itinerary

A small utility saves the AI's answer as a real Markdown file, so the plan can be reopened, printed, or shared later - instead of being lost the moment the chat window is closed.

Running and testing the full stack

The backend and frontend run as two separate commands (`uvicorn` for FastAPI, `streamlit run` for the UI), since they are genuinely independent programs. Real trip requests were then tested through the actual running app, not just the notebook - because new bugs can appear at the web-server and UI layer even when the underlying AI logic is completely correct.

7. All Bugs Found & Fixed (Simple Explanation)

These bugs were not guessed - each one was found by actually running the project's code and watching it fail, then fixed and re-tested to confirm the fix worked.

Bug 1: Diagram drawing could crash the whole request (main.py)

Problem: The step that draws a picture of the AI graph (my_graph.png) used the same try/except as the actual travel-planning logic. If that unrelated picture-drawing service was unreachable, the ENTIRE request failed with an error - even though the real question was never even answered.

Fix: Moved the diagram drawing into its own separate try/except, so it can only log a warning on failure and never blocks the real answer from being returned.

Bug 2: OpenAI model name was hardcoded (model_loader.py)

Problem: The code always used a fixed model name 'o4-mini' for OpenAI, completely ignoring whatever model name was actually written in config.yaml.

Fix: Changed the code to use the model name read from config.yaml, so changing the config actually works.

Bug 3: Broken currency conversion URL (currency_converter.py)

Problem: The web address used to fetch exchange rates had an extra slash by mistake (double slash), which can cause the request to fail or be misrouted.

Fix: Removed the extra slash so the address is built correctly.

Bug 4: App crashed instead of showing a clean error (streamlit_app.py)

Problem: When the backend failed to respond, the code tried to 'raise' a plain text message as if it were an error. Python does not allow that - it always crashes with a confusing 'TypeError' instead of showing a helpful message to the user.

Fix: Replaced it with a proper error message shown on screen (st.error), so failures are shown clearly instead of crashing the app.

Bug 5: Itinerary displayed as an unformatted grey block (streamlit_app.py & save_to_document.py)

Problem: The generated travel plan text had extra spaces at the start of every line. In Markdown, that many leading spaces means 'treat this as plain code', so the whole itinerary - headings, dividers, and all - showed up as one ugly grey code block instead of nicely formatted text.

Fix: Removed the extra leading spaces so the itinerary now displays as properly formatted text with real headings and dividers.

Bug 6: Silent search failures (place_search_tool.py)

Problem: The backup search (Tavily) only kicked in when Google Places completely crashed. If Google Places ran without error but simply returned nothing useful, the tool gave the AI no data at all and no backup was ever tried.

Fix: Rewrote the logic so the backup search also runs whenever Google returns an empty result, not only when it crashes.

Bug 7: Typo in the environment variable template (.env.name)

Problem: The template listed the Tavily key as 'TAVILAY_API_KEY' (two letters swapped by mistake). The actual code library only looks for 'TAVILY_API_KEY', so following the template exactly as written would silently disable the backup search - with no error shown anywhere.

Fix: Corrected the spelling in the template to match what the code actually reads.

Bug 8: The whole agent failed to start without a Google Places key (place_search_tool.py)

Problem: Building the AI agent tried to set up Google Places immediately, every single time, even if no Google key was provided. A missing or empty key made agent construction fail completely - blocking every single question, even ones that never needed place search at all.

Fix: Wrapped Google Places setup in its own safe check. If the key is missing, the app now just logs a warning and uses the free Tavily search instead, rather than failing to start.

Bug 9: Project secretly needed a paid API to run (.env.name / setup)

Problem: Google Places needs a Google Cloud billing account (a credit card on file) even though it has a monthly free credit - which goes against the goal of this project running 100% free.

Fix: Rewrote the environment template to clearly mark Google Places and OpenAI as optional/paid, and confirmed the app runs completely end to end using only 4 genuinely free keys: Groq, Tavily, OpenWeatherMap, and Exchange Rate API.

8. Tools & Models Used (Step by Step Flow)

This diagram shows exactly how the LLM and its tools are selected and connected: how config.yaml and ModelLoader pick a model provider, how that model is bound to four tool groups, and how each tool group maps to a real external API.

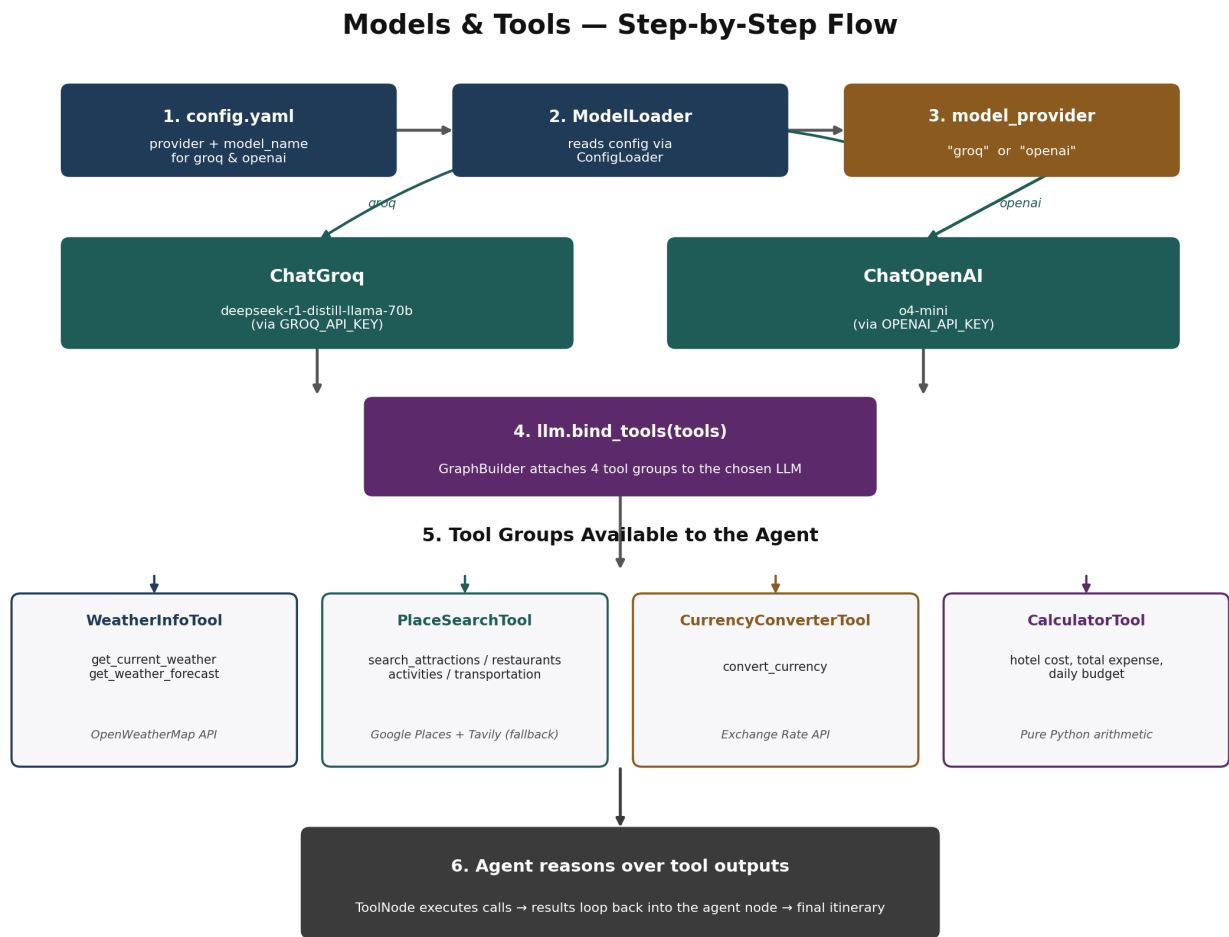


Figure 2: Step-by-step flow of model selection and tool binding.

9. Tips

- • Keep provider-specific logic (Groq vs OpenAI) behind one `ModelLoader`, so switching models later only means editing `config.yaml` - nothing else.
- • Always give an external-data tool a free backup (like Tavily backing up Google Places), so one API outage or missing key does not break the whole agent.
- • Test each tool alone in a notebook before adding it to the agent - it is far easier to debug one small function than a whole multi-step agent run.
- • Be very explicit in the system prompt about the exact output structure you want - vague prompts lead to inconsistent, unpredictable answers.
- • Visualize the compiled graph whenever the node/edge logic changes - wiring mistakes are much easier to spot in a picture than in code.
- • Never hardcode API keys - always use a `.env` file, and never upload that file to GitHub.
- • Choose free-tier APIs with no credit card requirement wherever possible, and build fallback logic so the project never depends on a paid service.
- • Test the full, real app (UI + API together), not just the underlying AI logic - some bugs only appear once every layer is actually being used together.

10. Resume Notes

Resume / LinkedIn bullet point

AI Trip Planner using LLMs & LangGraph - Built an intelligent, tool-using travel assistant with LangChain, LangGraph, FastAPI, and Streamlit that generates complete itineraries (attractions, restaurants, hotels, transport, budget, live weather) by dynamically calling real-time APIs instead of relying on static LLM knowledge. Designed a modular tool architecture with automatic fallback logic (Tavily backing up Google Places) for reliability, and decoupled the FastAPI backend from the Streamlit frontend for scalability.

One-line version

An agentic AI travel planner that turns a plain-language trip request into a full, real-data-backed itinerary - using LangGraph to let the AI decide when to check live weather, prices, and places on its own.

30-second spoken introduction

"I built an AI Trip Planner - it's an intelligent travel assistant that takes a simple request, like 'plan a 5-day trip to Goa,' and generates a complete itinerary automatically. Instead of just a regular chatbot, I used LangGraph to build an actual agent that can decide on its own when it needs to pull in real-time data - like current weather, live currency rates, or nearby attractions - before giving its final answer. It's powered by an LLM through Groq, exposed through a FastAPI backend, and has a simple Streamlit chat interface on top."

11. Interview Talking Points

For any 'why did you choose X' question, use this pattern: state the alternative you considered, state the concrete problem with it, then state why your choice solves that problem. Below are the key talking points for this project, in simple English.

Q: Why did you use an AI agent instead of a normal chatbot?

A: A normal chatbot only answers from memory, which goes stale (weather, prices change daily). An agent can call live tools to fetch real data before answering, so the answer is actually correct, not just guessed.

Q: Why LangGraph instead of writing your own if/else logic?

A: A real trip request does not clearly say which tools are needed. LangGraph lets the AI decide for itself, step by step, in a loop, instead of hardcoding every possible situation in advance.

Q: Why did you split the project into a backend (FastAPI) and frontend (Streamlit)?

A: So they can run and scale independently, and so the same backend could later power a different frontend (mobile app, another website) without duplicating any AI logic.

Q: Why did you give the AI a calculator tool - can't it just add numbers itself?

A: LLMs predict likely text, they don't truly calculate - they can make small arithmetic mistakes. A calculator tool guarantees the maths is always correct, which matters for a budget feature.

Q: Why does your place search use two providers (Google Places and Tavily)?

A: For reliability (one API going down doesn't break the feature) and for cost (Tavily is free, so the whole project can run without ever needing a paid Google Cloud account).

Q: Tell me about a real bug you fixed.

A: The graph-diagram drawing step called an external service, and if that service failed, it crashed the entire request - even though it had nothing to do with the actual travel answer. I isolated it into its own error handling so it can never block the real response.

Q: What are the current limitations of your project?

A: No memory across separate requests, no automated tests for the tools, no response streaming, and no persistent logging yet - all good next steps to mention as future improvements.

End of guide - AI Trip Planner using LLMs & LangGraph.